

Trazabilidad de requisitos en la evolución tecnológica del Museo Virtual STEAM: hacia un Sistema Recomendador de Cápsulas STEAM

Nicolás Rizzo^{1, 2, 3 y 4}, Daniela López De Luise^{1, 2, 3, 4 y 5} [0000-0003-3130-873X], Gilda R. Romero^{1, 2, 3 y 4} [0009-0002-3524-5967]

¹ CI2S Labs (Computational Intelligence & Information Systems Labs). Buenos Aires, Arg.

³ Sociedad Científica Argentina. Buenos Aires, Argentina.

² Centro de Altos Estudios en Tecnología Informática (CAETI). Universidad Abierta Interamericana - Facultad de Tecnología Informática. Buenos Aires, Argentina

⁴ Universidad Abierta Interamericana - Facultad de Tecnología Informática. Buenos Aires, Arg.
nicolasrizzo@gmail.com

Abstract. El presente trabajo aborda el desafío de resolver la trazabilidad de requisitos durante el proceso de migración y el condicionamiento de la plataforma "Museo Virtual STEAM" (MVS), orientado a establecer las bases para un Sistema Recomendador reglado y probabilístico. Entre otras cosas, se describe la trazabilidad de los requisitos durante la migración desde un entorno de gestión de contenidos hacia una arquitectura capaz de soportar la recomendación de "cápsulas" (unidades de aprendizaje STEAM). El núcleo de la propuesta radica en el uso de variables del entorno (sistema y servidor) para auditar y validar el porcentaje de uso de los requerimientos; se analizan las restricciones técnicas para justificar la baja o modificación de funcionalidades, proponiendo métricas específicas para la verificación y validación de requisitos. Este enfoque asegura que la plataforma migrada no solo preserve la integridad de los objetos de aprendizaje, sino que facilite la planificación y evaluación adaptada de actividades. Finalmente, se plantea el diseño para que el sistema sea escalable hacia Aplicaciones Web Progresivas (PWA) en una fase futura. La metodología empleada incluye ingeniería inversa de requisitos y un enfoque mixto para validar que la migración mantenga la integridad de los objetos de aprendizaje. Los resultados muestran que una trazabilidad apropiada permite detectar riesgos en la migración de una implementación y mejorar la calidad del proceso.

Keywords: Trazabilidad, requisitos de software, migración de Plataformas, STEAM, MVS, Sistema Recomendador.

1 Introducción

El modelo educativo STEAM se ha consolidado como un enfoque interdisciplinario que integra ciencia, tecnología, ingeniería, arte y matemáticas, con el propósito de desarrollar competencias vinculadas con la resolución de problemas, la creatividad, el pensamiento crítico y la aplicación práctica del conocimiento (Sanders, 2009; English, 2017; Bryan et al., 2015). A diferencia de los enfoques disciplinares tradicionales,

STEAM propone experiencias de aprendizaje integradas, donde los estudiantes pueden relacionar saberes diversos para abordar situaciones del mundo real (Hunter-Doniger & Sydow, 2016; Baran et al., 2021). La educación STEAM requiere herramientas que permitan socializar y monitorear actividades.

Como respuesta a la necesidad de centralizar y difundir este tipo de contenidos el Museo Sarmiento, surge la plataforma Museo Virtual STEAM (MVS) que difunde experiencias educativas interdisciplinarias a través de "Cápsulas", que son objetos de aprendizaje autocontenidos, dichos objetos educativos integran contenidos científicos, recursos didácticos y procesos específicos para facilitar experiencias inmersivas y replicables (López De Luise, 2021; López DeLuise et al., 2022).

La plataforma MVS es una herramienta tecnológica que funciona como un ecosistema donde se alojan y gestionan cápsulas de aprendizaje. Investigaciones previas sobre esta plataforma han permitido definir su ciclo de vida tecnológico y establecer mecanismos de explotación estadística para comprender el rendimiento de las actividades (Salina et al., 2022; Leiva, 2022) así como realizar exploraciones previas para trabajar como una Aplicación Web Progresiva (PWA) (Salina et al., 2024) y dar soporte a casos extremos de STEAM (Romero et al., 2023; Romero & López De Luise, 2025).

Actualmente, la herramienta MVS se encuentra en una fase de evolución crítica: la migración de su plataforma base para integrar un Sistema Recomendador reglado y probabilístico. El Sistema Recomendador (SR) busca facilitar a sus usuarios la selección de cápsulas de aprendizaje adaptadas a las condiciones específicas. Un sistema recomendador depende intrínsecamente de la pureza y consistencia de los datos transaccionales del usuario, si los registros de comportamiento son ambiguos o están corruptos, los algoritmos probabilísticos generarán recomendaciones espurias, frustrando el proceso educativo. Por ende, la necesidad de una plataforma robusta no es meramente arquitectónica, sino un requisito operativo crítico para la viabilidad del motor de recomendación. En tal sentido, para garantizar el éxito de esta transición, es imperativo un modelo de trazabilidad que resuelva la brecha entre los requisitos originales y la nueva lógica de recomendación, preparando el terreno para una futura implementación de PWA.

La migración de la plataforma MVS no es solo un cambio de arquitectura, sino una transición hacia la inteligencia colectiva a través del Sistema Recomendador. Sin embargo, la brecha entre la especificación teórica y el uso real en entornos vulnerables suele generar "deuda técnica" y pérdida de trazabilidad. Como señala Sommerville (2011), la ingeniería de requerimientos (IR) es el cimiento de cualquier proceso de migración de software y, particularmente, la trazabilidad es esencial para comprender el impacto del cambio y verificar que el sistema final cumpla con las expectativas.

En etapas previas de la versión "demo" de la plataforma propuesta por López De Luise et al. (2022), la gestión de requisitos se realizaba de forma discrecional, lo que dificulta la auditoría del cumplimiento funcional.

Este paper introduce un modelo de trazabilidad sistemático para la migración de la plataforma "Museo Virtual STEAM", evolucionando desde un prototipo funcional de la plataforma hacia una versión en otro servidor con agregado de un sistema recomendador de eventos (cápsulas STEAM en el contexto MVS) basado en un modelo reglado y probabilístico. El núcleo de la propuesta radica en el uso de

variables del entorno (sistema y servidor) para auditar y validar el porcentaje de uso de los requerimientos. Se analizan las restricciones técnicas para justificar la baja o modificación de funcionalidades, proponiendo métricas específicas para la verificación y validación de requisitos. Asimismo se evalúa el proceso para poder habilitar la incorporación del sistema recomendador de manera confiable. Este enfoque busca asegurar que la plataforma acondicionada sea el soporte robusto para el recomendador y su futura transición a Aplicaciones Web Progresivas (PWA).

2 Metodología

La práctica de la Ingeniería de software busca entregar a tiempo software operativo de alta calidad que contenga funciones y características que satisfagan las necesidades de todos los participantes (Pressman, 2010). Para lograrlo, se adopta un conjunto de principios o estrategias que permitan realizar la prueba del software y asegurar la calidad. En línea con esto, desde el punto de la Ingeniería de software, se adopta una estrategia de Validación y Verificación (V&V) para asegurar la calidad del software. En el contexto de una migración de plataforma, el V&V deja de ser una apreciación subjetiva para convertirse en un proceso de ingeniería basado en datos. Por su parte, la implementación de un modelo de trazabilidad genera información objetiva que, con el adecuado nivel de contexto y metadatos apropiados permiten una profunda comprensión de las decisiones y sus efectos durante el proceso migratorio. Tanto la matriz de trazabilidad como la V&V permiten objetivar la toma de decisiones durante la migración.

2.1 Validación y Verificación (V&V)

Siguiendo a Pressman (2010), la verificación se focaliza en responder a la pregunta “¿Estamos construyendo el producto correctamente?” a través del uso de métricas que permiten comprobar si el software migrado cumple con las especificaciones técnicas (latencia, carga de cápsulas, persistencia de datos). Por su parte, la validación, focalizada en responder a la pregunta “¿estamos construyendo el producto correcto?”, se basa en métricas de uso real (logs de navegación y feedback del usuario) quienes confirman si la plataforma realmente satisface las necesidades educativas.

2.2 Matriz de Trazabilidad

La Matriz de Trazabilidad de Requisitos (RTM, Requirements Traceability Matrix), propuesta por Karl Wiegers y Joy Beatty (2013), es una herramienta bidireccional que conecta las necesidades del negocio con los entregables del proyecto, el código y las pruebas, es decir, conecta cada requerimiento con sus fuentes de origen y con los artefactos de software resultantes del ciclo de vida (diseño, código, pruebas). Su objetivo es asegurar que no se pierda ningún requisito y evaluar el impacto de los cambios. Básicamente, consiste en una tabla que muestra los vínculos lógicos entre los requisitos funcionales individuales y otros artefactos del sistema tales como requisitos adicionales, de usuario, de negocio, cuestiones relacionadas con elementos de arquitectura y diseño, módulos de código, pruebas y reglas de negocio.

La RTM es una herramienta que facilita la gestión de los requerimientos a lo largo del ciclo de vida del proyecto de software y ayuda con el seguimiento al alcance, evaluación del impacto de cambios y los beneficios que se espera obtener.

La Tabla 1 detalla un ejemplo genérico de la estructura clásica de una RTM. Esta matriz permite la trazabilidad hacia adelante (forward) para asegurar que no se omitió ningún requerimiento durante la construcción, y hacia atrás (backward) para justificar la existencia de cada línea de código a partir de una necesidad real del negocio.

Tabla 1. Ejemplo de una entrada de RTM

ID Requisito	REQ-001			
Requisito de Usuario	Requisito Funcional (RF)	Elemento de Diseño / Arquitectura	Módulo de Código / Clase	Caso de Prueba (Test Case)
El sistema debe restringir el acceso a reportes según la jerarquía del usuario.	RF-01: El sistema validará los permisos de rol asignados antes de renderizar el dashboard de gestión.	Class SecurityManager Component: AdminDashboard	auth_service.py line 45: check_role()	TC-SEC-04: Intento de acceso a reportes por parte de un usuario con rol de invitado (Resultado esperado: HTTP 403).

2.3 Trazabilidad y V&V en la Ingeniería de Software

La integración de una matriz de trazabilidad con un protocolo de Validación y Verificación (V&V) constituye un imperativo dentro de la Ingeniería de Software, para garantizar la integridad transaccional de los requisitos durante procesos de migración compleja. Siguiendo los modelos de calidad de Boehm (1981) y McCall (1977), la trazabilidad se erige como el nexo causal que permite la observabilidad del software, transformando requisitos abstractos en métricas de cumplimiento verificables.

Desde la perspectiva de la calidad de software, McCall define factores como la *Corrección* (Correctness) y la *Fiabilidad* (Reliability) como pilares de un producto exitoso. En este sentido, la trazabilidad sistemática actúa como el mecanismo de control que asegura que cada funcionalidad migrada responda a un factor de calidad predefinido. Por su parte, Boehm enfatiza que la validación temprana y el seguimiento continuo de los requerimientos reducen exponencialmente los costos de reingeniería. En el contexto de sistemas adaptativos estos elementos no operan como meros artefactos de documentación, sino como componentes críticos de la arquitectura que permiten que la evolución de la plataforma sea un proceso determinístico y no estocástico ante las restricciones del entorno. Este trabajo extiende el concepto al contexto de la plataforma STEAM MVS.

3 Ingeniería Inversa de Requisitos

En el contexto del presente trabajo, la trazabilidad requiere una correcta y completa definición de variables del proceso, lo que demanda la selección de variables críticas para el funcionamiento esencial del sistema (el identificador del evento, de la actividad, comentarios, el identificador anonimizado del usuario y el rastro de actividades o log). Dado que la versión inicial de la plataforma carece de una especificación formal, se realiza un proceso de ingeniería inversa de requisitos complementado con ingeniería directa. La ingeniería inversa implica examinar un sistema existente para identificar sus componentes y crear representaciones en un mayor nivel de abstracción con el fin de deducir sus requisitos originales (Chikofsky & Cross, 1990). En esta línea, Sommerville (2011) señala que ante la ausencia de documentación, el código fuente se convierte en la única especificación confiable para derivar retrospectivamente las reglas de negocio. De este modo, la ingeniería inversa de requisitos permite abstraer los modelos de comportamiento de la versión “demo” para documentar qué funcionalidades se mantienen, modifican o descartan. Este análisis retrospectivo garantiza que el acondicionamiento de la arquitectura y la posterior evaluación de trabas técnicas protejan la integridad de las variables transaccionales que son fuente de forma determinística del sistema recomendador.

4 Desarrollo

El desarrollo de la plataforma MVS se rige por los estándares de Pressman (2010), quien enfatiza que la calidad del software depende de la concordancia con los requisitos funcionales explícitamente establecidos. El desarrollo de este trabajo se enfoca en cómo una migración impacta la trazabilidad de funcionamiento y calidad de la funcionalidad esencial que permite habilitar el recomendador.

4.1 Análisis retrospectivo e Ingeniería Inversa de Requisitos

Como parte del proceso de trazabilidad se realiza la identificación de los requisitos funcionales previos, es decir, de las cápsulas existentes en los documentos de referencia de López De Luise & Ruiz Tabarez (2019) y de Ramirez (2023) y el análisis comparativo contrastando el uso discrecional de la versión “demo” y el modelo de trazabilidad sistemático propuesto.

Se aplica ingeniería inversa de requisitos mediante la inspección de las tablas de la base de datos heredada y el análisis estático de los controladores PHP, derivando de forma retrospectiva el modelo conceptual original para poder plasmarlo en la RTM actual.

4.2 Definición de variables de Trazabilidad

En el contexto del presente trabajo, la trazabilidad requiere una correcta y completa definición de variables de trazabilidad del proceso donde también se requiere la selección de variables críticas para el funcionamiento esencial (en el caso de MVS,

las detectadas son el id del evento, de la actividad, comentarios sobre la actividad, id anonimizado del usuario o persona que realiza la actividad, y el rastreo de actividades también conocido como log). Se realiza una **ingeniería de requisitos inversa y directa** para documentar qué funcionalidades se mantienen, se modifican o se descartan durante el proceso de acondicionamiento para el sistema recomendador y la arquitectura, se realiza una evaluación de funcionalidades que presentan trabas técnicas para decidir su baja o reingeniería.

4.3 Mapeo de Requisitos: Elaboración de la Matriz de la Trazabilidad

En el proyecto de migración y acondicionamiento tecnológico (desde un prototipo funcional básico hacia un prototipo con sistema recomendador) y con perspectiva para una arquitectura PWA), la Matriz de Trazabilidad Sistemática (RTM) tradicional se adapta puesto que ya no solo mapea "código", sino que conectan requerimientos funcionales adaptativos con variables de telemetría de servidor para realizar procesos continuos de Validación y Verificación (V&V).

La matriz integra, de forma exhaustiva, los Requerimientos Funcionales (RF), relacionándolos con las variables de contexto del usuario (ubicación, recursos disponibles) de entorno específicas para conectar los atributos de las cápsulas (complejidad, materiales, tiempo). Las variables detalladas en la Tabla 2 (*id persona*, *Log*, *nivel de dificultad*) no solo actúan como rastros para una auditoría técnica sino que constituyen las dimensiones del espacio vectorial que alimentará el modelo probabilístico del recomendador. La trazabilidad sistemática asegura que cada una de estas variables de telemetría esté directamente vinculada a un Requisito Funcional (RF) específico, garantizando que el comportamiento capturado en el servidor corresponda exactamente a una acción educativa prevista y no a un ruido del entorno. En la tabla 2 se detalla un ejemplo de dicha matriz considerando 5 de los requerimientos involucrados.

Tabla 2 - Ejemplos de entradas de Matriz de Trazabilidad MVS

ID Req	Requerimiento Funcional (RF)	Variables de Entorno (Evidencias)	Tipo de Control
RF-01	El sistema debe permitir al usuario seleccionar una o más condiciones de contexto xSTEAM (Bajo acceso tecnológico, ubicación, etc.).	id persona, rol, establecimiento (Persona), elemento (CTA) (Log), edad (derivada), nivel de dificultad	Validación
RF-02	El sistema debe recomendar cápsulas mostrando una lista con las cápsulas STEAM	id persona (Persona), url, elemento (CTA) (Log), cápsula realizada (Evento).	Validación

RF-03	El sistema debe permitir calificar cápsulas aplicadas y completar feedback con comentarios.	parecer, aprendizaje, aplicabilidad, otra participación, experiencia favorita (Comentarios).	Verificación
RF-04	El sistema debe permitir descargar cápsulas y recursos en formatos accesibles (PDF).	elemento (CTA), url (Log), id persona (Persona).	Validación
RF-05	El sistema debe mostrar a los gestores las cápsulas más usadas por región o qué brechas existen (ej. falta de contenido lingüístico).	cápsula realizada (Evento), país, región, ciudad (Log), url (Log).	Validación

El mapeo de requisitos como el RF-03 en la RTM facilita la futura transición hacia esquemas PWA, puesto que permite aislar qué funcionalidades de feedback y descarga de objetos de aprendizaje deben ser gestionadas mediante Service Workers y estrategias de almacenamiento en caché en escenarios de conectividad limitada (visto en RF-04).

4.4 Estudio de caso

En el marco de la ingeniería inversa y el acondicionamiento del "Museo Virtual STEAM" (MVS) para soportar el modelo recomendador, se analiza un escenario real de restricción técnica que impacta directamente en la trazabilidad y la reconfiguración de los requisitos del sistema. Durante las tareas de mantenimiento, generación de copias de seguridad (*backups*) y replicación del sitio hacia el entorno de desarrollo local protegido (*sandbox*), Se pudo detectar un comportamiento anómalo en el núcleo de la aplicación provocado por la persistencia del código malicioso *Backdoor Etomidetka* (InCyberEra, 2024).

La amenaza hallada es una técnica avanzada de evasión orientada a vulnerar la integridad de la plataforma: generaba perfiles de administrador ocultos y no indexables desde el panel de gestión nativo del CMS¹ (Wordfence, 2023), lo que tiene como objetivo impedir su auditoría visual directa. Mediante el análisis de los archivos fuentes PHP y las tablas de la base de datos MySQL en el entorno local, es posible identificar que el vector de entrada del atacante se produjo a través de vulnerabilidades en complementos (*plugins*) de terceros desactualizados.

¹ Un CMS (por sus siglas en inglés Content Management System) es un Sistema de Gestión de Contenidos, una herramienta de software que permite crear, editar, organizar y publicar contenido digital en sitios web sin necesidad de conocimientos técnicos avanzados o programación. Wordpress está categorizado como un CMS.

El impacto de la amenaza Etomidetka va más allá de la seguridad del CMS; ataca directamente la médula del Sistema Recomendador. Al generar perfiles ocultos y manipular registros, el malware corrompe las variables de entorno `id persona` y `Log` (ver Tabla 2), inyectando datos estocásticos maliciosos que habrían sesgado por completo el entrenamiento del motor probabilístico. Es aquí donde la ingeniería inversa de requisitos demuestra su valor: permitió contrastar el comportamiento anómalo del servidor con la especificación original, aislando el código corrupto y protegiendo la integridad de las variables de recomendación antes de la migración final.

Este incidente exige un proceso de remediación y auditoría que impacta en la matriz de requisitos del paper mediante las siguientes acciones:

1. **Depuración y baja de funcionalidades:** se procede a la eliminación inmediata de falsos *plugins* inyectados y de todos aquellos complementos prescindibles. Esto justifica técnicamente la baja y modificación de ciertos requisitos de usuario en pos de reducir la superficie de ataque y garantizar la seguridad del entorno.
2. **Saneamiento y validación de persistencia:** se purgan manualmente los registros huérfanos en las tablas `wp_users` y `wp_options` y se regeneran las claves de sal (*salt keys*) en el archivo `wp-config.php` para invalidar sesiones concurrentes no autorizadas.

En tal sentido, el incidente demuestra un conflicto entre un Requisito No Funcional referido a Seguridad (RNF-Seguridad) y los Requisitos Funcionales adaptativos (ej. RF-01, RF-02). La solución requirió un proceso de evolución de requisitos y desapropiación (Descoping) de funcionalidades. En términos numéricos, por ejemplo, originalmente en la demo se identifican más de 15 requerimientos, de los cuales 3 requerimientos debieron darse de baja por trabas técnicas. De esta manera, los resultados demuestran que una trazabilidad reglada reduce la incertidumbre en la implementación y garantiza la calidad de las cápsulas STEAM.

El estudio de este caso convalida empíricamente la necesidad de utilizar variables del entorno y del servidor **para auditar y depurar los logs transaccionales frente a accesos anómalos o registros corruptos**. Este proceso de endurecimiento (*hardening*) asegura que la base de datos migrada preserve la integridad de los objetos de aprendizaje y que el futuro motor híbrido de recomendación se alimente exclusivamente de interacciones generadas por usuarios legítimos en un entorno informático estable.

4.5 Discusión

La migración orientada a sistemas recomendadores permite una personalización que los museos virtuales tradicionales no poseen. La transición hacia un modelo sistemático permite reducir la incertidumbre en la migración. La comparación con la versión previa revela que la trazabilidad basada en variables de entorno permite una toma de decisiones informada sobre qué componentes priorizar. Se discute cómo estas

métricas facilitan la identificación de "trabas" técnicas, permitiendo que la negociación de requisitos con los stakeholders (docentes y alumnos) se base en datos reales de rendimiento y no en supuestos.

5 Conclusiones

La resolución de la trazabilidad durante la migración es el pilar que permite transformar una plataforma de exposición pasiva en un sistema activo de recomendación.

La implementación de métricas de trazabilidad sistemática transforma el proceso de migración en una actividad auditable y eficiente. El uso de variables de sistema para validar el porcentaje de uso de requerimientos garantiza que el Sistema Recomendador de cápsulas STEAM se asiente sobre una plataforma técnicamente verificada.

Este estudio presentado, evidencia que la correcta trazabilidad habilita una calidad mínima funcional ya que no solo resuelve la trazabilidad actual, sino que define un estándar para la evolución del Museo Virtual hacia entornos PWA más resilientes. Se concluye que el diseño actual deja el camino libre para una transición fluida hacia las tecnologías necesarias garantizando la sostenibilidad tecnológica a largo plazo.

Referencias bibliográficas

1. Baran, M., Baran, M., Karakoyun, F., & Maskan, A. (2021). The influence of project-based STEM (PjbL-STEM) applications on the development of 21st century skills. *Journal of Turkish Science Education*, 18(4), 798–815. <https://doi.org/10.36681/tused.2021.104>
2. Bryan, L. A., Moore, T. J., Johnson, C. C., & Roehrig, G. H. (2015). Integrated STEM education. In C. C. Johnson, E. E. Peters-Burton, & T. J. Moore (Eds.), *STEM road map: A framework for integrated STEM education* (pp. 23–37). Routledge.
3. Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall. (Referente en economía y calidad de software).
4. Chikofsky, E. J., & Cross, J. H. (1990). Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1), 13-17. <https://doi.org/10.1109/52.43044>
5. English, L. D. (2017). Advancing elementary and middle school STEM education. *International Journal of Science and Mathematics Education*, 15(Suppl 1), 5–24. <https://doi.org/10.1007/s10763-017-9836-z>
6. Hunter-Doniger, T., & Sydow, L. (2016). A journey from STEM to STEAM: A middle school case study. *The Clearing House: A Journal of Educational Strategies, Issues and Ideas*, 89(4–5), 159–166. <https://doi.org/10.1080/00098655.2016.1170461>
7. InCyberEra. (2024, 18 de enero). WordPress backdoor: Unmasking the Etomidetka threat. <https://incyberera.com/blog/wordpress-backdoor-unmasking-the-etomidetka-threat/>
8. López De Luise, D., & Ruiz Tabarez, E. A. (2019). *STEAM: nueva Autopista del Aprendizaje*. ISBN 978-620-0-05687-0, 978-620-0-05687-0. EAE. 2019.
9. López De Luise, D. (2021). *Museo inmersivo STEAM* [Tesis de grado, Universidad de Buenos Aires, Facultad de Ciencias Exactas y Naturales].

10. López De Luise, D., González González, V., Draier, E., Salina, C., Leiva, S. M., & Ramírez, S. (2022). Experiencias educativas desde la comunidad del museo histórico Sarmiento. Dinámicas de una cultura científico-tecnológica inmersiva. *Revista Avatares de la Comunicación y la Cultura* (23).
11. McCall, J. A., Richards, P. K., & Walters, G. F. (1977). *Factors in Software Quality*. General Electric. (Referente histórico del modelo de calidad de software).
12. Pressman, R. S. (2010). *Software Engineering: A Practitioner's Approach* (7th ed.). McGraw-Hill.
13. Ramírez, S. (2023). Tesis: LINCIEVIS, plataforma de cápsulas STEAM. [Tesis de grado: Licenciatura en Sistemas informáticos. 2021 – 2023, Universidad Abierta Interamericana (UAI)]
14. Romero, G. R., & López De Luise, D. (2025). xSTEAM en acción. *JAIIO, Jornadas Argentinas de Informática*, 11(8), 259-268. <https://revistas.unlp.edu.ar/JAIIO/article/view/19962>
15. Romero, G. R., Salina, C., Domen, M., López De Luise, D., & Bejarano, M. (2023). Generación de STEAM en condiciones extremas con xSTEAM. En *Libro de actas – XXIX Congreso Argentino de Ciencias de la Computación (CACIC 2023)* (pp. 662–671). Universidad Nacional de La Plata. <https://sedici.unlp.edu.ar/handle/10915/165076>
16. Salina, C. V. (2023). *Plataforma digital de cápsulas STEAM: ciclo de vida STEAM, integrando aplicaciones web progresivas*. [Tesis de grado, Universidad Abierta Interamericana]. <https://repositorio.uai.edu.ar/handle/123456789/2272>
17. Salina, C., Dome, M., López De Luise, M. D., Rizzo, N. & Romero, G. R. (2024). PWA para potenciar una plataforma STEAM. En el XXX Congreso Argentino de Ciencias de la Computación (CACIC) (La Plata, Argentina, 7–11 de octubre de 2024). <https://sedici.unlp.edu.ar/handle/10915/176338>
18. Salina, C., López De Luise, D., Leiva, S., Ramírez, S., & Romero, G. R. (2022). A life-cycle to improve STEAM virtual museum activities exposition. En *10th International Workshop on Soft Computing Applications* (Arad, Romania, 21–23 November 2022).
19. Sanders, M. (2009). STEM, STEM education, STEMmania. *Technology Teacher*, 68(5), 20–26. https://esdstem.pbworks.com/f/TTT+STEM+Article_1.pdf
20. Sommerville, I. (2011). *Software Engineering* (9th ed.). Pearson Education.
21. Wiegers, K., & Beatty, J. (2013). *Software Requirements*. Microsoft Press.
22. Wordfence. (2023, 22 de noviembre). How to clean a hacked WordPress site using Wordfence. <https://www.wordfence.com/docs/how-to-clean-a-hacked-wordpress-site-using-wordfence/>